NUS-ISS

MASTER OF TECHNOLOGY IN SOFTWARE ENGINEERING

Graduate Certificate Examination
Sample Examination Paper

Subject: *Architecting Scalable Systems*

Solutions

SECTION A

Question 1

(Total: 25 marks)

a. Other answers will be considered in their own merits.

i. Below is a possible reason. Other reasons would be considered on their merits.

A frequently forgotten condition for load testing (considered a kind of performance testing) is the workload model (e.g., whether the load is constant throughout, stepped up over time, varied, etc.) If the few testers had made different assumptions and employed different workload models, the test results would certainly be different. To conduct a proper test, it is essential to specify the workload model as part of the test criteria.

(2 Marks)

ii. Below are two possible reasons. Other reasons would be considered on their merits.

Firstly, for the *Update Current Simplified/Detailed Results* use case to produce deterministic and repeatable results, it is essential for the "voting" events to be temporally ordered. A swap in the order of two events would change the results temporally. This calls for the use of the event streaming mechanism as it ensures the ordering of events. Secondly, the use case is required to update both the simplified as well as detailed results; this means it actually have to go through a sequence of new "voting" events twice. This also calls for the use of the event streaming mechanism as it would not drop its events within certain time window (vs. the message queue mechanism which drops events upon delivery).

(2 Marks)

Marking Scheme for (i):

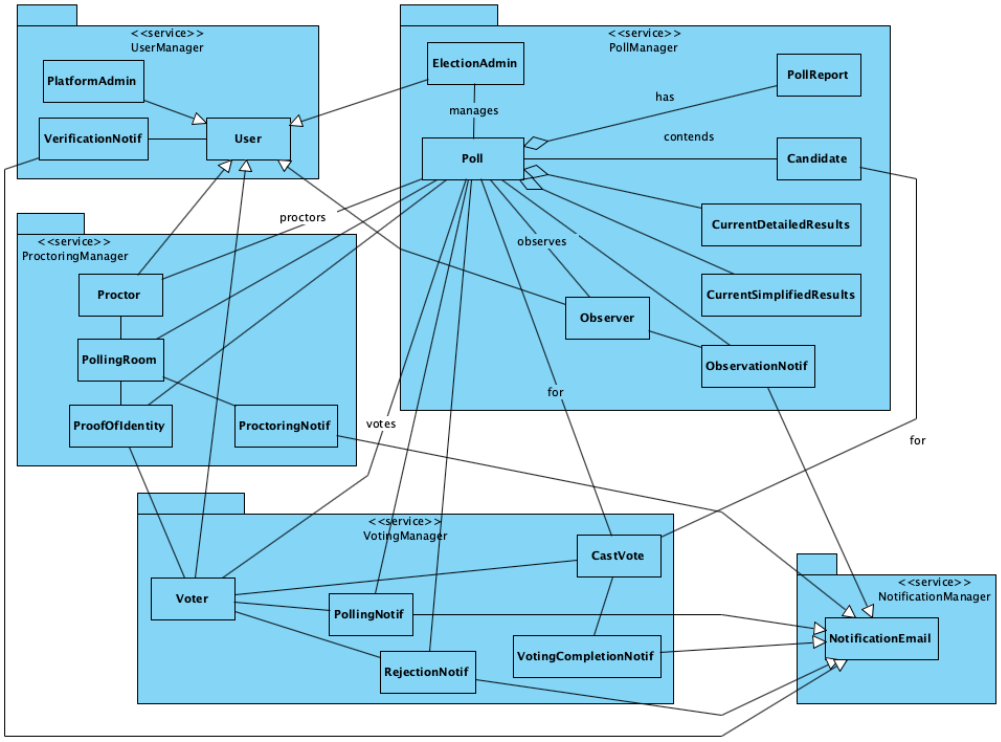
- *1 mark for suggesting a possible reason.*
- *1 mark for proposing a more reliable way.*
- *Up to a maximum of 2 marks.*

Marking Scheme for (ii):

- *1 marks for suggesting each of two possible reasons.*
- *Up to a maximum of 2 marks.*

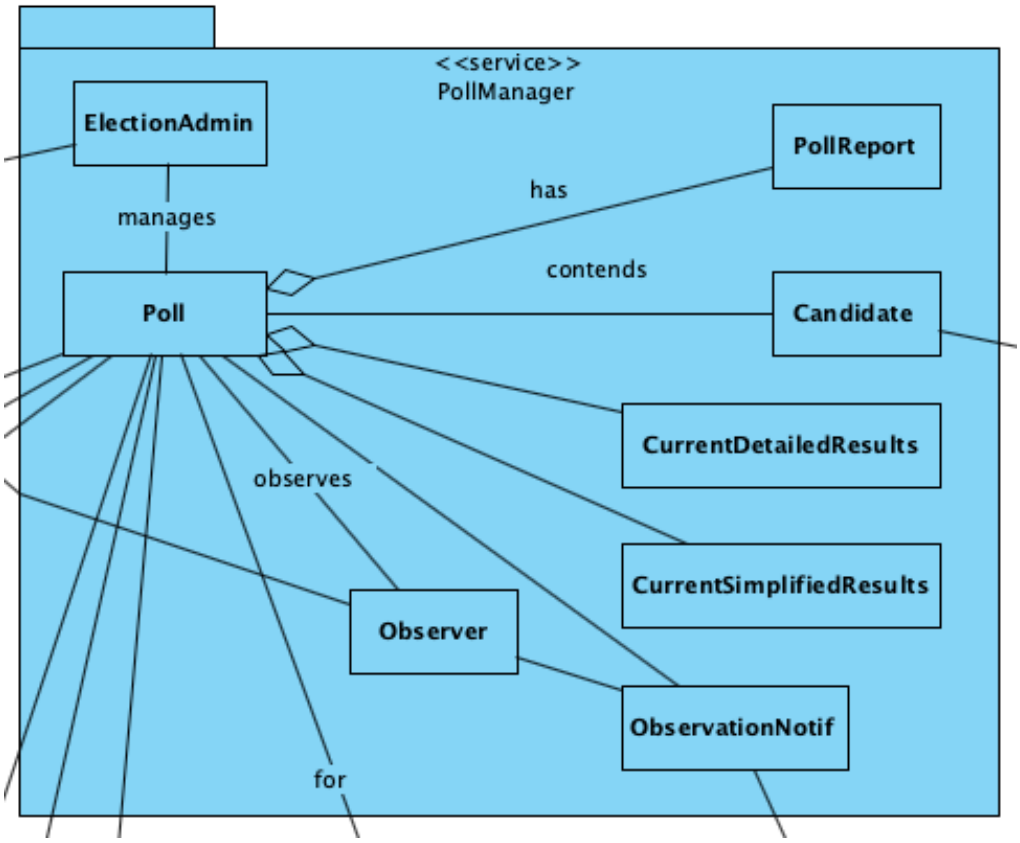
b. Issues that arise due to the domain model.

i. The current domain design is following the theme of collating all the related functions in the same service. For instances, the various kinds of notification are handled in the NotificationManager service. While the design approach may appear to be neat, the NotificationManager service handles all the specificities of the various kinds of notification. These specificities would need to be removed from the service if the latter is to be made reusable for other platforms. The issue can be mitigated by migrating the specificities from the NotificationManager service to the other services as illustrated by the diagram below. As a result, the refactored NotificationManager service would no longer include the specificities which are not reusable. This makes the service highly reusable across other platforms.



Doing the above does not result in any major drawback. In fact, the separation of notifications based on their kinds allows their corresponding services to be scaled independently as well.

ii. At present, the viewing of CurrentSimplifiedResults of a Poll by an Observer would require collaboration between the PollManager and ObservationManager services. Since inter-process communication, synchronous or event-based, would be required whenever there are interactions between the two services, it is reasonable to expect such interactions to take much longer than intra-process communication. This issue can be mitigated by migrating the entities of the ObservationManager service to the PollManager service, as illustrated by the diagram below. This would certainly improve the promptness of the Observer in viewing the updated CurrentSimplifiedResults of a Poll.



Doing the above could potentially result in a relatively large service (i.e., bounded context) but the side effects (e.g., large transaction scope) may be mitigated by having multiple aggregates within the service.

Marking Scheme for (i):

- *2 marks for the most likely cause of the issue.*
  1.5 marks for the countermeasure for the kind-specific functional segregation of the issue.
  1.5 marks for the countermeasure for the kind-generic functional cohesion of the issue.
- *0 mark for the drawback of the countermeasure of the issue.*
- *Up to a maximum of 5 marks.*

Marking Scheme for (ii):

- *1.5 marks for the most likely cause of the issue.*
  1.5 marks for the countermeasure of the issue.
- *1 marks for the drawback of the countermeasure of the issue.*
- *Up to a maximum of 4 marks.*

(9 Marks)

c. The possible answers are as follows. Other possible explanation and answers will be considered in their own merit.

1. It doesn't comply with RMM Level 2, which advocates using different verbs based on the HTTP request methods. In the interfaces described, HTTP POST verb

should be used to submit an entity to the specified resource, to effect a change in state of the resource on the server.

2. API versioning is important. To support continuous changes for published APIs without creating unnecessary problems for existing API consumers, URI versioning might not be the best approach. If it is not well-documented or managed carefully, it can likely cause client code to break when a version is deprecated or made obsolete. Using a HTTP Request custom header (i.e., Accept-version) will be a better solution as this will preserve a clean set of URI endpoints, where content negotiation (i.e., versioning) is done through the request headers.

3. The proposed interfaces as illustrated:

URI	**Service**	**Inputs**	**Outputs**	**Explanation**	**HTTP Action**
/votes/invite	ProctoringManager		Voter-id	Invite the next voter to the polling room for verification	POST
/votes/{voter-id}/start	ProctoringManager	Voter Identifer	List of candidates of the poll	Put a voter to voting mode	POST

Marking Scheme:

- *2 marks for each of the 3 parts to provide relevant explanation and articulating the alternative design.*
- *Up to a maximum of 6 marks.*

(6 Marks)

d. One possible answer is to highlight the importance of performance and security requirement for this system.

- Performance or scalability is very important and challenging given that there's a requirement to support 100,000 concurrent voters.
- Due to the scenario, security is very important especially the guarantee that each voter can only vote maximum once.

Marking Scheme:

- *Up to 1 mark for each convincing explanation on the most important non-functional requirement.*

(2 Marks)

e. The question is quite open-ended, but any accepted answers must have these characteristics: 1) it is measuring something related to the outcome of the system, which is accurate voting and usage of the system, 2) there is an argument why measuring this is beneficial in improving matching effectiveness. Some possible answers:

- Average Duration of Identity Verification: it is beneficial for system scalability to allow for faster identify verification. A quicker identity verification also indicates that the system is more user friendly and provides better support for identity verification by the proctor.
- Average Time for Voting per voter: track and measure the usability of the system and performance of the system to support the main use case of the system.
- Average Queueing Time per voter: track and measure voter's experience and the system scalability to support the voting

Marking Scheme:

- *2 marks for each proposed metrics and its justification.*
- *Up to a maximum of 4 marks.*

(4 Marks)

Question 2

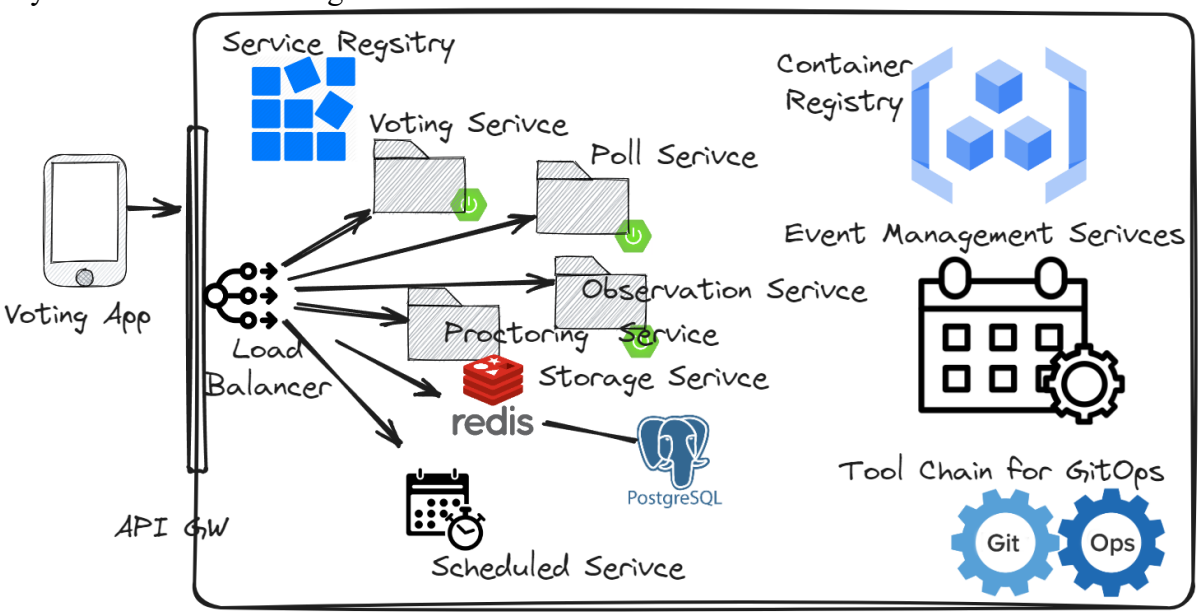
(Total: 13 marks)

(This rubric provides a model answer. Rubric will consider alternative ways to conceptualise architecture based on the merits, demerits and constraints necessary.)

a. Hybrid reference architecture.

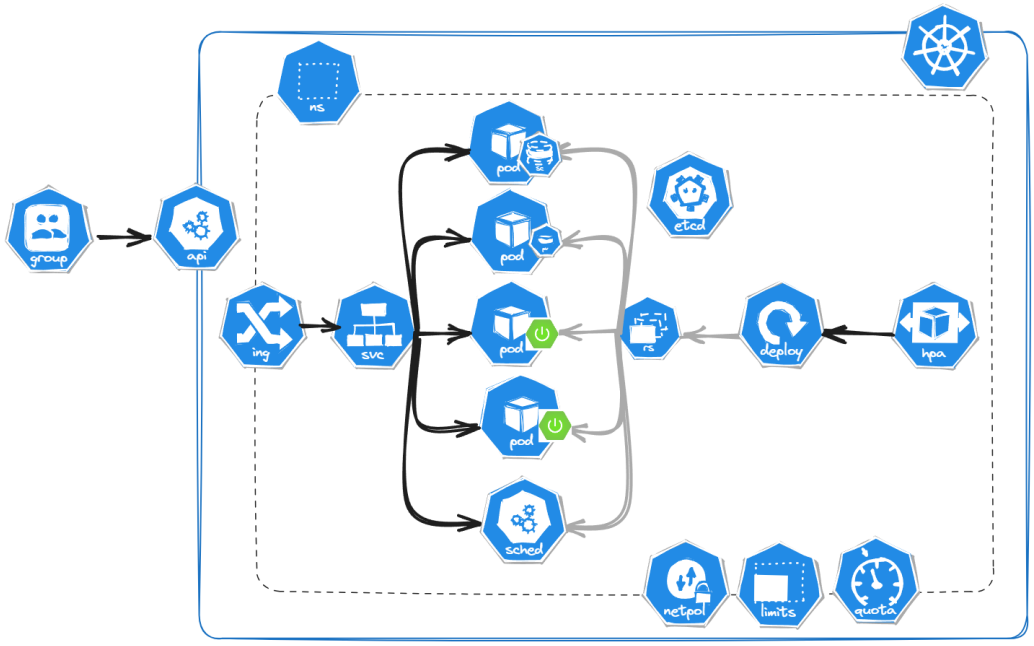
The platform backend can be composed of microservices, scheduled jobs and event driven architecture. Given the volume and metric discussions in Section 4 of the case study, there is sufficient volume requirement to choose microservices for the platform backend. The backend services can be visualized using five different services (as illustrated below). A simple distributed application running across multiple containers in a Kubernetes environment.

Physical Architecture Diagram



A microservice-based architecture is illustrated in the following diagram, representing a real-time voting application, where small microservices process and consolidate user votes. The voting application collects individual user votes from polling devices frontend. The backend voter collects the vote in an in-memory redis solution and there on persists in a postgres datastore. The poll service keeps track of status via in-memory datastore. Also the proctoring service would serve the other web front end via collected statistics. Finally, there is application logic in the observation service and scheduled tasks, which aggregates all of the voting data cast by users and returns the final results. To support the eco-system of microservices, event and scheduled jobs envisioned we need git ops tool chain and container registries. Also to manage the API engineering aspects, service registry, load balancers, ingress translators and istio meshes are useful. The Kubernetes Pod abstraction for running thousands of containers in a flexible manner has been a fundamental part of the transition to containers in the voter application. Application delivery involves upgrading, downgrading, parameterizing,

and customizing many different Kubernetes resources. The overall k8s deployment can be visualized as below



Rubric: 4 marks for reference architecture (identify services, registries, state mechanism and storage), 2 marks for additional communication components including event services and notification services; 1 mark for identifying cache or in-memory services; 3 mark for deployment plan identifying pods, deployment and services.

(10 Marks)

b. Front End Design.

(i) In client-side state management, data is managed in the user's web browser using techniques such as cookies, local storage, and session storage. This allows for fast and efficient data retrieval and manipulation, but can be limited by the storage capacity of the user's device and potential security concerns. In server-side state management, data is managed on the web server, allowing for more robust data storage and retrieval capabilities. This can include techniques such as container managed state, caching, and server-side database. Both Progressive web app and native app can implement application wide state management. PWA's use selectors and stores. While native applications use library and framework support. Thus, the design choices are indifferent from storage perspective.

(ii) However, it is specified that voting application plans to native services from IoT based sensors and actuators in the polling stations for users with health conditions and special needs. Hence building a native mobile front end is justified in the *Vote Anywhere* platform.

(iii) Here client routing is preferred and sharing state can be managed at client side as well. By registering multiple API dependencies can be resolved. No changes required to back end services.

Rubric: 3 marks in total; 1 mark for discussing storage option in NA vs PWA; 1 mark for identifying need for native app need and relevant explanation. 1 mark for

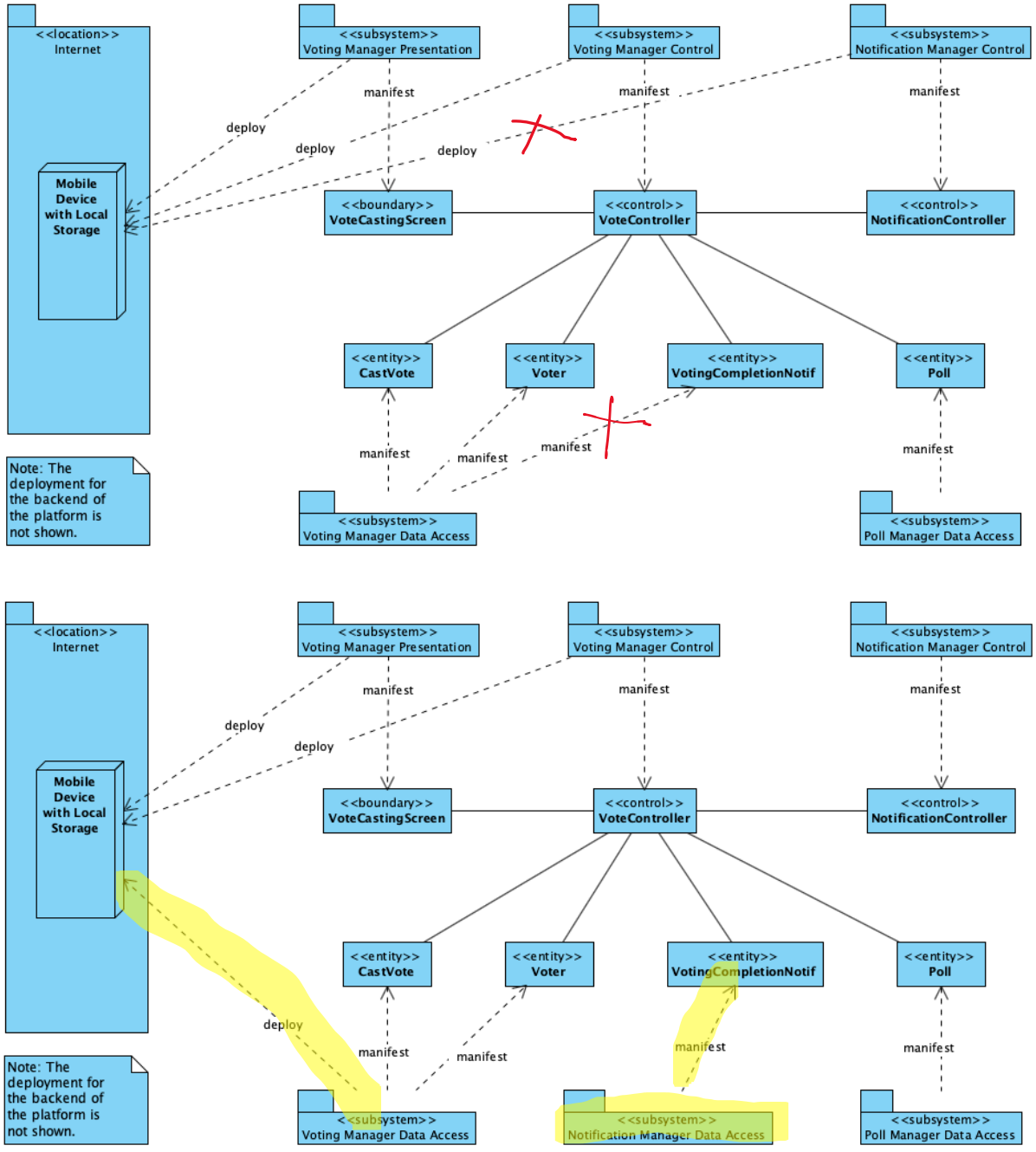
explaining that the client-side orchestration is preferable for this case and no change in back end services.

(3 Marks)

Question 3

(Total: 12 marks)

a. Example of a more desirable logical packaging cum deployment model along with the issues marked up in the original model. Other reasonable issues would be considered accordingly.



The following are examples of undesirable architectural decisions and their corrective actions:

- The VotingCompletionNotif entity should not be packaged into the Voting Manager Data Access subsystem. It should be packaged into the Notification Manager Data Access subsystem.
- The Voting Manager Data Access subsystem should be deployed to the mobile device as the latter needs to persist the CastVote entity on its local storage.
- The Notification Manager Control subsystem should not be deployed to the mobile device as notifications are not sent from the frontend.

Marking Scheme:

- *1 mark for each of the two undesirable architectural decisions*
- *1 mark for each of the two corrective actions*
- *1 mark for the proper justification of each of the two corrective actions*
- *Up to a maximum of 6 marks*

(6 Marks)

b. The question is testing understanding about scaling the database and its effect on the overall architecture.
The sharding should be done based on the voters using some kind of unique voter identification (e.g. voter ID, NRIC, passport number) and optionally the system can allocate different set of proctor for every voter shards.

The components that have to be shard aware are:

- VoteController: to ensure that data are recorded in the right shard
- PollManager: to ensure that poll are recorded in the right shard
- UserManager: because users are stored in different shards
- ObservationManager: need to aggregate across shards

Marking Scheme:

- *Up to 2 marks for the explanation on how the sharding can be done.*
- *Up to 4 marks for pointing out the affected components and the justification.*

(6 Marks)